Professional Training in Cybersecurity I

Module 0 — Session 2

The Working Environment and the Submission Workflow

Mohamed Gebril

Session Objective

By the end you can do each of these — on your own machine, not just on the slides:

Install — the four tools, inside Kali
and say what each one is for, so you never wonder which to open

Write — a worksheet in markdown
headings, fenced code, a screenshot that actually appears

Record — your work with Git
a commit for each exercise, and `git status` to see where things stand

Authenticate — to GitHub with your key
the same pair from last session, a second lock, no password

Submit — a full cycle, end to end
from pulling the session to a push GitHub confirms

Diagnose — a submission that did not land
the Git messages that stop a submission, and the fix for each

Session Agenda

Part 1 — The Environment in the Machine

why the tools live inside Kali, and the one job each of the four has

Part 2 — VS Code

a folder, a terminal that starts in it, and the extensions that add a language

Part 3 — Markdown

source and rendered view, every mark the worksheets use, and the settings that write image links

Part 4 — Flameshot

a screenshot as evidence, and saving it where the worksheet looks

Part 5 — Git

what Git records, how it differs from GitHub and GitLab, the commands that move a change, and this course's two repositories

Part 6 — Authenticating to GitHub

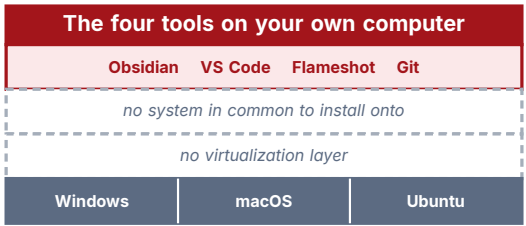
the same pair, a different lock, and why the clone URL decides everything

Part 7 — The Submission, End to End

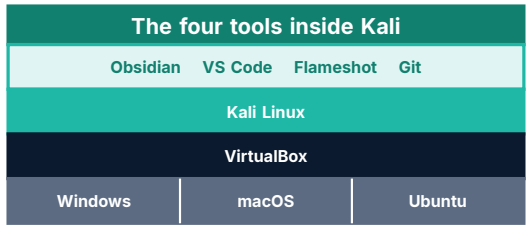
starting and finishing a session, and the mistakes that stop a submission

The Environment in the Machine

Why the Tools Are Installed Inside Kali



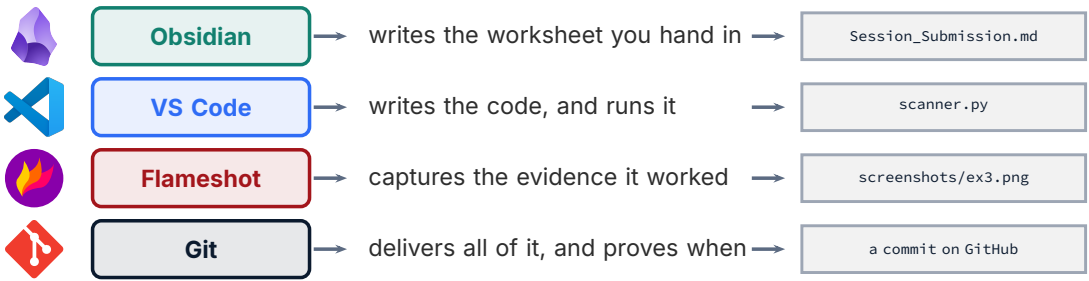
installed three different ways, so **three sets of instructions**



installed once, so **one set of instructions**

- **Every tool this course uses is installed inside the Kali machine, not on the computer in front of you.**
- Students run three different host systems. One environment means one set of instructions, which is the leveling this module exists to deliver.
- It also means a broken environment costs a snapshot restore rather than an evening, and nothing installed for this course can disturb anything you already had.
- **Containers run in here too**, later, which keeps that work off your own machine and away from the host settings that fight with VirtualBox.

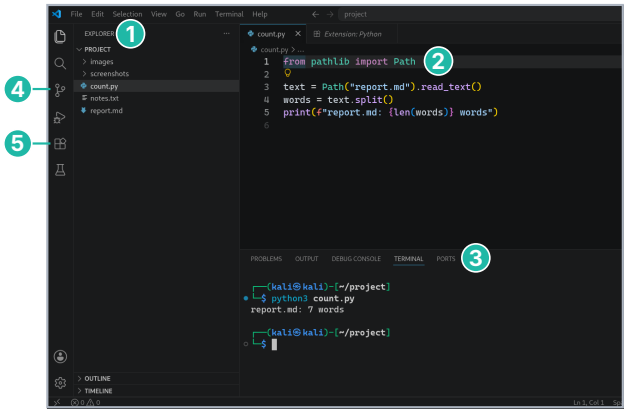
Four Tools, and the One Job Each Has



- **Obsidian is a markdown editor.** It works on a folder of plain files, so the file you edit is the file you hand in.
- **VS Code is a code editor with a terminal built in**, so a script and the output it prints stay in one window.
- **Flameshot is a screenshot tool.** It captures part of the screen and saves it straight to a file you name.
- **Git is a version control system.** It records a snapshot of your folder, with a time and a message, and sends it to GitHub.

VS Code

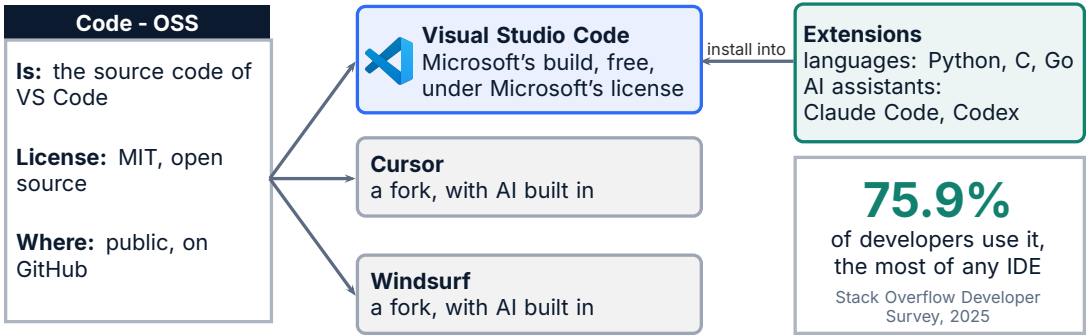
What an IDE Is



1. **Explorer:** the folder's files, as in a file manager
2. **Editor:** the code, colored by language
3. **Terminal:** a shell, opened in the folder
4. **Source Control:** Git, inside the window
5. **Extensions:** add languages and tools

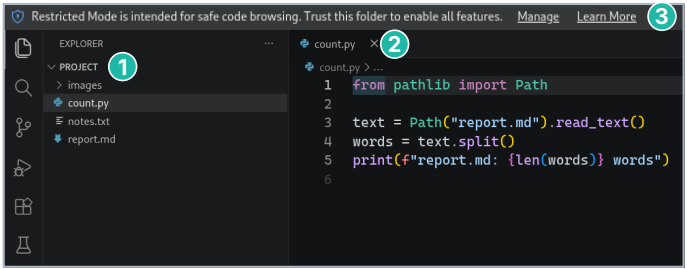
- **An IDE, an integrated development environment, puts the tools for writing code in one window**: files, editor, terminal and Git.
- **Microsoft calls VS Code a code editor.** Extensions add the language support that makes it a full IDE.

Why This Course Uses VS Code



- **Its source code is open, under the MIT license.** The VS Code you install is Microsoft's build of that code: free, but under Microsoft's own license.
- **Cursor and Windsurf are forks of it, and AI assistants such as Claude Code and Codex install into it**, so what you learn here carries over.

Opening a Folder, Not a File

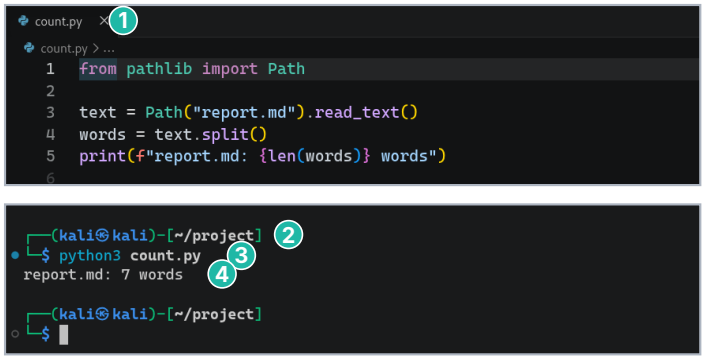




1. **Explorer:** every file in the opened folder
2. **Editor tab:** the file open for editing
3. **Restricted Mode:** until you trust it
4. **Trust:** only for your own folders

- **VS Code opens a folder, not a single file.** The Explorer lists everything in it, so the whole project is one click away.
- **A new folder opens in Restricted Mode**: click **Manage**, then **Trust**, only for a folder you made or got from the course.

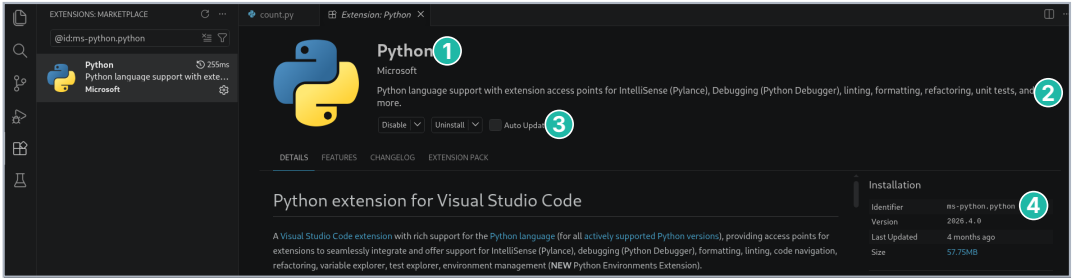
The Built-In Terminal Starts in the Folder



1. **Editor:** `count.py`, the script to run
2. **Prompt:** `~/project`, the opened folder
3. **Command:** `python3 count.py`
4. **Output:** what the script printed

- **Terminal, New Terminal opens a shell inside VS Code**, already in the folder you opened.
- **`python3` and `git` act on the folder the prompt shows**, so read it before you type.

Extensions Add What an Editor Lacks



1. **Name:** Python, published by Microsoft
2. **Description:** what it adds to the editor
3. **State:** installed; disable or uninstall it here
4. **Identifier:** `ms-python.python`, unique on the Marketplace

- **VS Code knows no programming language by itself.** An extension adds one: the Python extension adds highlighting, running and error checking.
- **Check the publisher and the ID before you install.** Look-alike extensions exist, and only the ID is unique.

Markdown

Markdown: Source Text and Its Rendered View

source: the characters saved in the file

```
## Exercise 2: The NAT Network
(blank line)
Both machines answered **after** the build.
(blank line)
- the gateway answered
- the second machine answered
```

rendered view: what Obsidian draws

Exercise 2: The NAT Network

Both machines answered **after** the build.

- the gateway answered
- the second machine answered

- **Markdown is a markup language: formatting is written as ordinary characters in the text.** A hash sign (`#`) marks a heading and a dash (`-`) marks a list item.
- The saved file is the **source**. Obsidian **renders** it: it reads the marks and draws the formatted view without changing the file.
- **A blank line ends a paragraph.** Keep one between each paragraph, heading and list.

Why Worksheets Are Markdown, Not Word Documents

a markdown file, opened as plain text

```
$ cat worksheet.md

## Exercise 2: The NAT Network

Both machines answered **after** the build.

- the gateway answered
- the second machine answered
```

a Word document, opened as plain text

```
$ cat worksheet.docx

PK□h8]_rels/.rels□□MKA□□□C□□l+□□□□
"Bo"□□□□□□□3i□□□A□P□□□y□□□m□□□N□□□
A□iAq0□0jx□=/`□/□W>□□J□\*□□□aI□□□L
□41q□□!fOR□<b"□□□q□□□2□□1□□j□[□□□H
```

... 7,949 bytes of this in all

- **A markdown file is plain text: any program shows the words you wrote.** The grader can read it, search it, and copy commands out of it.
- **A Word document is not plain text.** Word stores the words in a compressed format, so only a word processor can show them.
- **A word processor also changes characters as you type**, such as two hyphens into one dash, so a command copied out of it stops working.

Headings, Emphasis, and Escaping a Mark

source	rendered view

`# Session 2 Submission`
`## Exercise 2`
`### Step 1`
	Session 2 Submission
	Exercise 2
	Step 1

`**both** machines answered` `the *second* machine`	**both** machines answered the *second* machine
`the result is 5\*3\*7`	the result is 5*3*7

- **A heading is a line that starts with hash signs and a space.** The number of hashes is the level: one for the title, two for an exercise, three for a step.
- **Two stars make text bold and one star makes it italic.** Put the stars directly against the words they wrap.
- **A backslash before a mark prints the mark itself**, so a star you want to show is typed as `\*`.

Lists: Markers, Nesting, and Checkboxes

source

```
- the gateway answered
- the second machine answered
  - after the network was built

1. run the command
2. capture the output
   - paste both into the worksheet

- [ ] captures are in screenshots/
- [x] all three checks pass
```

rendered view

- the gateway answered
- the second machine answered
  - after the network was built

1. run the command
2. capture the output
   - paste both into the worksheet

- ☐ captures are in screenshots/
- ☑ all three checks pass

- **A dash starts an item in a list, and a number with a period starts an item in a numbered list.** Use numbers only when the order matters.
- **To nest an item, indent it to where the text above it starts**: two spaces under a dash, three under `1.`. Each dot on the left is one space.
- **A dash, a space and `[ ]` make a checkbox.** Type `x` between the brackets to tick it.

Code: Inline Spans and Fenced Blocks

source

````
Run `ssh lab` from Kali.

```bash
$ ssh lab whoami
student
```

```python
for machine in ["kali", "ubuntu"]:
    print(machine, "answered")
```
````

rendered view

Run `ssh lab` from Kali.

```
$ ssh lab whoami
student
```

```
for machine in ["kali", "ubuntu"]:
    print(machine, "answered")
```

- **Backticks mark code inside a sentence**, so a command name reads as code and not as an ordinary word.
- **Three backticks on a line of their own open a code block, and three more close it.** The word after them names the language: `bash` for commands, `python` for Python.
- **A code block keeps every character and space**, so code copied out of it runs as written. Keep a command and its output in one block.

Links and Images: One Character Apart

source	rendered view
`[GitHub Docs](https://docs.github.com)`	GitHub Docs
`[ex2](screenshots/ex2.png)`	ex2
`![ex2](screenshots/ex2.png)`	 the picture itself

- **A link is text in square brackets followed by an address in round brackets.** The text becomes clickable.
- **An image is the same notation with an exclamation mark in front**, and the picture itself appears in its place.
- **The words in the square brackets are the alternative text**, shown when the picture cannot be displayed. Name it for the exercise it proves.

Tables: The Header Row and the Delimiter Row

source

```
| Machine | Account |
| ------- | ------- |
| Kali    | kali    |
| Ubuntu  | student |
```

```
|Machine|Account|
|---|---|
|Kali|kali|
|Ubuntu|student|
```

rendered view

Machine	Account
Kali	kali
Ubuntu	student

Machine	Account
Kali	kali
Ubuntu	student

- **A table is a header row, a delimiter row of dashes, and then the data rows.** The delimiter row is what makes the first row a header.
- **The pipes do not need to line up.** Both sources on this slide render the same table.
- **Leave a blank line before and after a table.** GitHub reads a line typed directly under it as another row.

Quotes, Callouts, and Dividers

source

```
> Keep screenshots beside this file.

> [!note] How to use this submission
> This file is your graded artifact.

Task 1 is done.
(blank line)
---
(blank line)
```

rendered view

> Keep screenshots beside this file.

> **How to use this submission**
> This file is your graded artifact.

Task 1 is done.

- **A line that starts with a greater-than sign is a quote.** Type a space after the sign, and start every line of the quote with it.
- **A callout is a quote whose first line starts with a type in square brackets, such as `[!note]`.** Obsidian draws it as a colored box, with the rest of that line as its title.
- **Three dashes alone on a line draw a divider.** Leave a blank line above and below them.

One Worksheet Section, Typed and Rendered

source

```
## Warm-up: Check the lab — REQUIRED
**The check, from Kali:**
```bash
$ ssh lab whoami
student
```
> [!example] Evidence — screenshot
> *(the check — `warmup-ssh.png`)*
(blank line)
![warmup-ssh](screenshots/warmup-ssh.png)
```

rendered view

Warm-up: Check the lab — REQUIRED

The check, from Kali:

```
$ ssh lab whoami
student
```

Evidence — screenshot
(the check — `warmup-ssh.png`)



the screenshot itself

- **A worksheet section is a prompt, your pasted output, and the evidence callout.**
- **Put the screenshot on its own line below the callout, after a blank line.** The blank line ends the callout.
- **Check the file in Obsidian's reading view before you push.** Reading view shows it rendered, with no marks.

Screenshots: The Folder and the Relative Path

the folder on disk

```
Module0-Foundation/
└── Session2/
    ├── Module0_Session2_Practical_Guide_Submission.md
    ├── Module0_Session2_Home_Task_Submission.md
    └── screenshots/
        ├── warmup-ssh.png
        └── one capture for each graded item
```

how the link finds the file

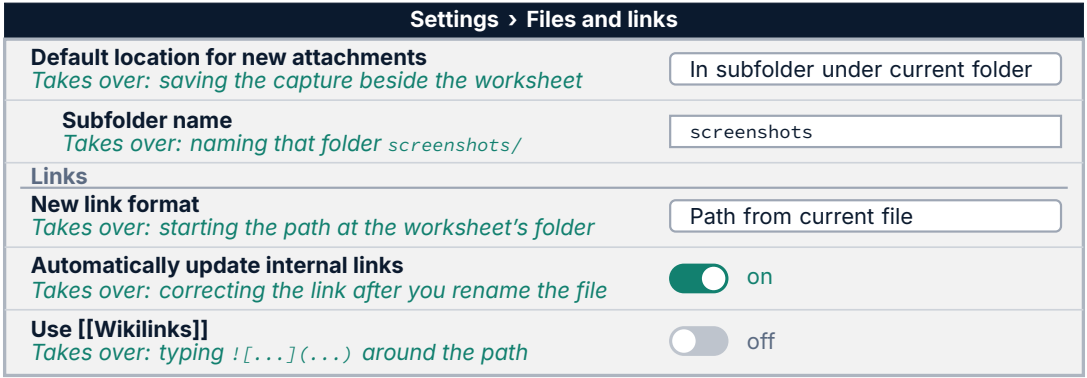
The worksheet is in
`Module0-Foundation/Session2/`

Its image link says
`screenshots/warmup-ssh.png`

So the file it opens is
`Module0-Foundation/Session2/screenshots/warmup-ssh.png`

- **An image path is read from the folder that holds the worksheet.** Each slash goes one folder deeper.
- **Keep one `screenshots/` folder beside the worksheets, and push it together with them.**
- **Name each capture for the item it proves, in lowercase, with hyphens and no spaces.**

Obsidian Settings: Writing the Link Automatically



- **Set these once in each vault.** *Subfolder name* appears once the option above it is chosen.
- **Paste a screenshot, and Obsidian saves it and writes the link.** Its name is *Pasted image* plus the time.
- **Two steps stay yours:** rename the file for its item, and type that name in the square brackets.

Common Mistakes: Headings, Emphasis, and Lists

what was typed	what it shows	the correct form
`#Results`	#Results — *plain text*	`#·Results`
`**·both**`	** both** — *the stars stay*	`**both**`
`5*3*7`	537 — *the 3 is italic*	`5\*3\*7`
`-check the network`	-check the network — *no list*	`-·check the network`
`1. Start Kali` `··- check the network`	1. Start Kali • check the network — *two lists*	`1. Start Kali` `···- check the network`

`Task 1 is done.`
`---`
	Task 1 is done. — *a heading*	`Task 1 is done.`
		(blank line)
		`---`

- **Put a space after `#` and after `-`, and let emphasis stars touch the words.**
- **Stars in ordinary text need a backslash**, or the text between them turns italic.
- **A nested item under a number needs three spaces, and a divider needs a blank line above it.**

Common Mistakes: Code Blocks and Tables

what was typed	what it shows	the correct form

``bash
$ ssh lab whoami
student
(blank line)
## Task 2 	
	$ ssh lab whoami
	student
	
	## Task 2
	the heading is code
		``bash
		$ ssh lab whoami
		student
		``
		(blank line)
		## Task 2

| Machine | Account | | Kali | kali |	| Machine | Account | | Kali | kali | *no table*	| Machine | Account | | --- | --- | | Kali | kali |

- **Close every code block with three backticks on a line of their own.** Everything after an open block is shown as code.
- **A table needs its delimiter row**, the row of dashes under the header.

Common Mistakes: Images and Screenshots

what was typed, and the correct form	what it shows
`![ex1](screenshots/ex*1.png)` ***correct:*** `![ex1](screenshots/ex1.png)`	**ex** *no image: the path stops at the space*

`> [!example] Evidence — screenshot`
`> *(the check — `ex1.png`)*`
`![ex1](screenshots/ex1.png)`
correct: a blank line above the image line
	Evidence — screenshot
	(the check — ex1.png)
	 
	inside the callout

`![ex1](ex1.png)` ***correct:*** `![ex1](screenshots/ex1.png)`	 *Obsidian finds it by name, the checker does not*
`![[ex1.png]]` ***correct:*** `![ex1](screenshots/ex1.png)`	 *Obsidian shows it, the checker reports it*

- **Name capture files with no spaces**, and leave a blank line between the callout and the image.
- **Write the path from the worksheet's folder**, even when Obsidian finds the file without it.
- **Use the standard image link, not `![[...]]`**, so the submission checker can find the picture.

Flameshot

A Screenshot Is Evidence



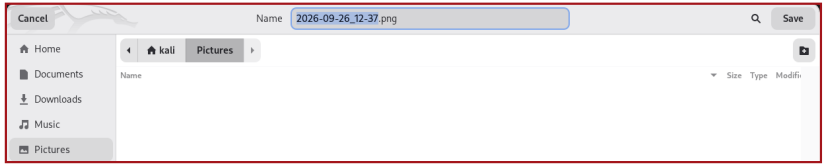
1. **Prompt:** user, machine and folder
2. **Command:** `python3 count.py`
3. **Result:** the output line, boxed in red
4. **Save:** writes the capture to a file
5. **Dimmed area:** outside the region, not saved

- **Capture the prompt, the command and its output in one region**, and box the line that proves the result.
- **Never capture a private key, a password or a token.** A screenshot is pushed to GitHub and stays in its history.

Saving It Where the Worksheet Looks

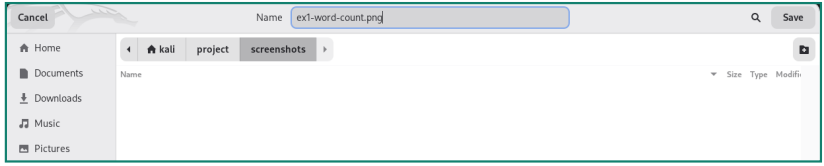
Flameshot's default

`~/Pictures`, named by timestamp: no link finds it



Saved for the worksheet

`~/project/screenshots`, named `ex1-word-count.png`



- **Flameshot saves to Pictures with a timestamp name** unless you change both.
- **Save into the session's `screenshots` folder with the suggested name**, so the worksheet's link `screenshots/ex1-word-count.png` finds it.

Git

Git, GitHub, and GitLab

a program on your machine	websites that store Git repositories	
Git	**GitHub**	**GitLab**
		
Is: a program that records the history of a folder.	**Is:** a website that stores Git repositories.	**Is:** a website that stores Git repositories.
Runs: on your own machine, even offline.	**Runs:** on GitHub's servers.	**Runs:** on GitLab's servers, or on an organization's own.
Made by: Linus Torvalds and the Linux developers, 2005.	**Made by:** GitHub, part of Microsoft since 2018.	**Made by:** GitLab Inc.

- **Git is a program that records the history of a folder**, on your own machine.
- **GitHub and GitLab are websites that store copies of Git repositories**, to share them and hand them in.
- **Git works on its own, without either website.** A website adds a copy that other people can reach.

What a Commit Is

one commit: a snapshot of every file you have added to Git, saved with an ID, an author, a time and a message

the folder

```
project/
  report.md
```

`git commit` →

the commit

ID	`0e78837`
author	Omar Khaled
time	Fri Sep 25, 11:31
message	Write the first draft
files	`report.md`

kali@kali:~/project$

```
$ git add report.md                                  # pick the file
$ git commit -m "Write the first draft"              # take the snapshot
  [main (root-commit) 0e78837] Write the first draft # first commit, ID 0e78837
   1 file changed, 3 insertions(+)
   create mode 100644 report.md
```

- **Instead of saving a copy of the file for each version**, you commit, and Git keeps the snapshot.

Your Name on Every Commit

your Git settings, saved once

name	Omar Khaled
email	omar.khaled@example.com

copied into →

every commit you make

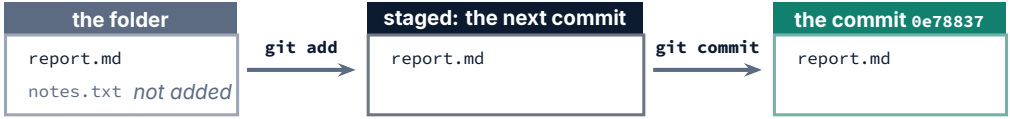
ID	`0e78837`
author	Omar Khaled
email	omar.khaled@example.com
time	Fri Sep 25, 11:31
message	Write the first draft

kali@kali:~$

```
$ git config --global user.name "Omar Khaled"                    # set your name
$ git config --global user.email omar.khaled@example.com         # set your email
$ git config --global --list                                     # read them back
  user.name=Omar Khaled
  user.email=omar.khaled@example.com
```

- **Git writes your name and email into every commit**, so each snapshot says who made it.
- **`--global` saves them for every repository on this computer**, so the two lines are run once, not once per folder.

Choosing What Goes Into a Commit



kali@kali:~/project$

```
$ git add report.md                                    # choose one file
$ git status                                           # check what is chosen
  On branch main

  No commits yet

  Changes to be committed:                             # chosen for the next commit
    (use "git rm --cached <file>..." to unstage)
          new file:   report.md

  Untracked files:                                     # not chosen, left out
    (use "git add <file>..." to include in what will be committed)
          notes.txt
```

- `git add` **chooses which files go into the next commit.** A file that is never added stays out of every commit.

Checking Where Things Stand

kali@kali:~/project$

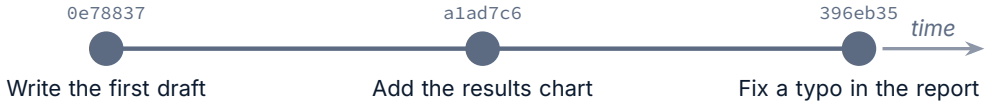
```
$ git status                                          # after making the chart
  On branch main
  Untracked files:                                    # new, not added yet
    (use "git add <file>..." to include in what will be committed)
          images/
          notes.txt

  nothing added to commit but untracked files present (use "git add" to track)
$ git add images/                                     # choose the chart
$ git commit -m "Add the results chart"
  [main a1ad7c6] Add the results chart                # the second commit
   1 file changed, 0 insertions(+), 0 deletions(-)
   create mode 100644 images/chart.png
$ git status                                          # and again after committing
  On branch main
  Untracked files:
    (use "git add <file>..." to include in what will be committed)
          notes.txt                                   # still left out, on purpose

  nothing added to commit but untracked files present (use "git add" to track)
```

- `git status` **shows where each file is, and changes nothing.** Run it before and after every commit.

The History: A Chain of Commits



kali@kali:~/project$

```
$ git log --oneline                                  # every commit, newest first
  396eb35 (HEAD -> main) Fix a typo in the report    # HEAD -> main: the newest
  a1ad7c6 Add the results chart
  0e78837 Write the first draft
```

- **Each commit follows the one before it**, so the commits form a history of the folder.
- **A repository is a folder together with all of its commits.**

Local and Remote: Two Copies of One Repository



local repository
on your computer

the network

remote repository
on GitHub





you make commits here

others reach them here



```
kali@kali:~/student-YOUR-USERNAME$

$ git remote -v                                                    # where the remote is
  origin  git@github.com:ORG/student-YOUR-USERNAME.git (fetch)     # origin: its name
  origin  git@github.com:ORG/student-YOUR-USERNAME.git (push)
```

- **A local repository is the copy on your own computer.** You make commits there.
- **A remote repository is a copy on a server such as GitHub**, where other people can reach it.

Clone: Copying a Remote Repository Once



kali@kali:~$

```
$ git clone git@github.com:ORG/student-YOUR-USERNAME.git
  Cloning into 'student-YOUR-USERNAME'...                # a new folder, same name
  remote: Enumerating objects: 10, done.
  remote: Counting objects: 100% (10/10), done.
  remote: Compressing objects: 100% (6/6), done.
  remote: Total 10 (delta 0), reused 10 (delta 0), pack-reused 0 (from 0)
  Receiving objects: 100% (10/10), done.                 # every commit is now here
```

- **Clone copies a remote repository onto your computer**, with all of its commits. You clone a repository once.
- This command reaches GitHub, so it works once your SSH key is added to GitHub, in the next part.

Push: Sending Your Commits



kali@kali:~/student-YOUR-USERNAME$

```
$ git push                                                  # after committing 774bb0a
  Enumerating objects: 5, done.
  Counting objects: 100% (5/5), done.
  Delta compression using up to 8 threads
  Compressing objects: 100% (3/3), done.
  Writing objects: 100% (3/3), 356 bytes | 356.00 KiB/s, done.
  Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
  To github.com:ORG/student-YOUR-USERNAME.git                # where it went
     396eb35..774bb0a  main -> main                         # GitHub now has 774bb0a
```

- **Push sends your new commits to the remote repository.** Until then, only you can see them.
- This command reaches GitHub, so it works once your SSH key is added to GitHub, in the next part.

Pull: Receiving New Commits

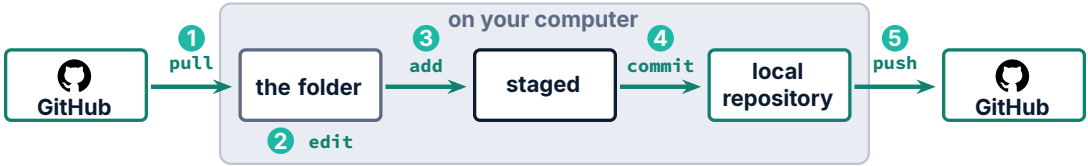



```
kali@kali:~/student-YOUR-USERNAME$

$ git pull                                              # the instructor pushed feedback
  remote: Enumerating objects: 4, done.
  remote: Counting objects: 100% (4/4), done.
  remote: Compressing objects: 100% (3/3), done.
  remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
  Unpacking objects: 100% (3/3), 371 bytes | 371.00 KiB/s, done.
  From github.com:ORG/student-YOUR-USERNAME             # where it came from
     774bb0a..3b9410a  main       -> origin/main
  Updating 774bb0a..3b9410a                             # your copy catches up
  Fast-forward
   feedback.md | 3 +++                                  # the instructor's feedback
   1 file changed, 3 insertions(+)
   create mode 100644 feedback.md
```

- **Pull brings the remote's new commits onto your computer.** Pull before you start work.
- This command reaches GitHub, so it works once your SSH key is added to GitHub, in the next part.

The Workflow: Pull, Edit, Add, Commit, Push

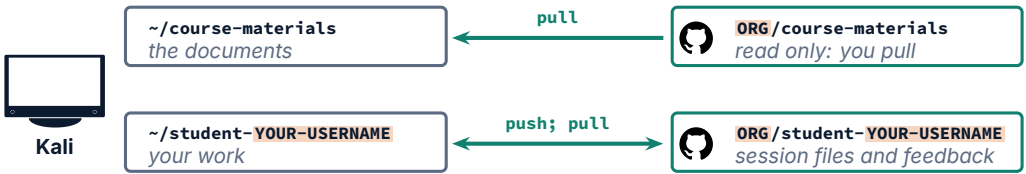


kali@kali:~/student-YOUR-USERNAME$

```
$ git pull                                  # 1. start from the newest commits
# 2. edit report.md in your editor, and save
$ git add report.md                         # 3. choose what goes in
$ git commit -m "Add the conclusion"        # 4. snapshot, still only here
$ git push                                  # 5. now it is on GitHub
```

- **Pull, edit, add, commit, push**: the same five steps every time you work in a repository.
- **A change is on GitHub only after the push.** In the first three places, it is on your computer alone.

Git in This Course




```
kali@kali:~/student-YOUR-USERNAME$

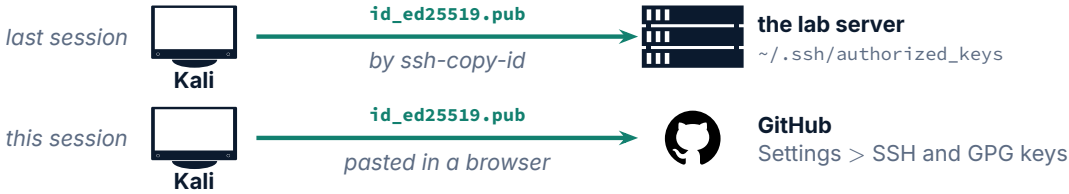
$ tree Module0-Foundation/Session1                    # one finished session
  Module0-Foundation/Session1                         # mirrors course-materials
  ├── Module0_Session1_Home_Task_Submission.md
  ├── Module0_Session1_Practical_Guide_Submission.md
  └── screenshots                                     # one per session
      ├── ex3-kali-desktop.png
      └── task2-plain-nat.png

  2 directories, 4 files
```

- **Your repository is private**: only you and the instructor can see it.
- **`ORG` is your course's organization**: `Professional-Training-Cybersecurity-I`.

Authenticating to GitHub

The Same Key Pair, a Different Lock

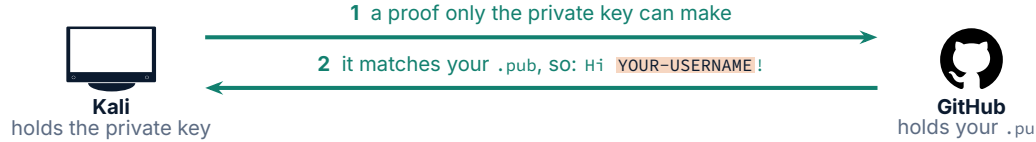


kali@kali:~$

```
$ ls -l ~/.ssh/id_ed25519*
  -rw------- 1 kali kali 411 Sep 26 15:32 /home/kali/.ssh/id_ed25519          # private
  -rw-r--r-- 1 kali kali  99 Sep 26 15:32 /home/kali/.ssh/id_ed25519.pub      # public
```

- **Last session's key pair now proves who you are to GitHub.** Only the `.pub` half is copied.
- **GitHub has no shell for `ssh-copy-id`**, so the `.pub` line is pasted into your account's settings.

Proving GitHub Accepts Your Key



kali@kali:~$

```
$ ssh -T git@github.com                                    # the first time only
  The authenticity of host 'github.com (140.82.121.4)' can't be established.
  ED25519 key fingerprint is: SHA256:...                   # compare with GitHub's list
  This key is not known by any other names.
  Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
  Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
  Hi YOUR-USERNAME! You've successfully authenticated, but GitHub does not provide
   shell access.                                           # success, not an error
```

- **The first connection asks you to confirm GitHub's key**: check the fingerprint against GitHub's list.
- **A greeting with your username means it worked.** GitHub gives no one a shell.

Why the Clone URL Decides Everything

The HTTPS URL

`https://github.com/ORG/student-YOUR-USERNAME.git` → **asks for a username and a token**

```
Username for 'https://github.com':
```

The SSH URL

`git@github.com:ORG/student-YOUR-USERNAME.git` → **uses your key: nothing to type**

```
~/.ssh/id_ed25519
```

```
kali@kali:~/student-YOUR-USERNAME$

$ git remote -v                                                    # which URL?
  origin  https://github.com/ORG/student-YOUR-USERNAME.git (fetch)
  origin  https://github.com/ORG/student-YOUR-USERNAME.git (push)
$ git remote set-url origin git@github.com:ORG/student-YOUR-USERNAME.git   # switch to SSH
$ git remote -v
  origin  git@github.com:ORG/student-YOUR-USERNAME.git (fetch)
  origin  git@github.com:ORG/student-YOUR-USERNAME.git (push)
$ git pull                                                         # no password
  Already up to date.
```

- **The URL decides whether Git uses your key.** An HTTPS URL asks for a token instead.
- **Copy the SSH URL from the repository's Code button**, and fix an older copy with `git remote set-url`.

The Submission, End to End

Starting a Session: Pull Both Repositories

```
kali@kali:~$
$ cd ~/course-materials
$ git pull                                          # the instructor published Session 2
   [16 lines: progress, and one line per new file]
    7 files changed, 2844 insertions(+)             # the session's documents
   [7 "create mode" lines]
$ cd ~/student-YOUR-USERNAME                        # your repository
$ git pull                                          # the session's folder arrives
   [14 lines: progress, and one line per new file]
    5 files changed, 1389 insertions(+)             # guide, home task, worksheets
   [5 "create mode" lines]
$ ls Module0-Foundation/Session2
   Module0_Session2_Home_Task.md
   Module0_Session2_Home_Task_Submission.md
   Module0_Session2_Practical_Guide.md
   Module0_Session2_Practical_Guide_Submission.md
   screenshots
```

- **Pull both repositories before you start**: yours brings the session's folder and your feedback.
- **Fill in only the two worksheets**: the guide and the home task beside them are there to read.

Finishing a Session: Add, Commit, Push

```
kali@kali:~/student-YOUR-USERNAME$
$ git add Module0-Foundation/Session2                    # the session folder only
$ git commit -m "Module 0 Session 2: exercise 4"
  [main f5e9418] Module 0 Session 2: exercise 4
   2 files changed, 6 insertions(+), 4 deletions(-)      # the worksheet, one screenshot
  [1 "create mode" line: the screenshot]

$ git push
  [7 lines: counting and writing 766 KiB]
  To github.com:ORG/student-YOUR-USERNAME.git
     229c38f..f5e9418  main -> main                      # three commits, now on GitHub
$ git status                                             # check once more
  On branch main
  Your branch is up to date with 'origin/main'.          # nothing left to push

  nothing to commit, working tree clean
```

- **Commit each finished worksheet, and push before the deadline**: only what is on GitHub then counts.
- **Run `git status` last**: `up to date with 'origin/main'` means GitHub has everything.
- **Never commit a password, key or token**: Git keeps every commit, so deleting it later does not unpublish it.

Common Mistakes: Committing

```
$ git commit -m "Module 0 Session 1: exercise 3"     # no name and email set
  Author identity unknown
  [11 lines: how to set them]
  fatal: unable to auto-detect email address (got 'kali@kali.(none)')
# fix: set them once with git config --global, then commit again
```

```
$ git commit -m "Module 0 Session 1: exercise 3"     # committed without add
  [8 lines: the branch, and the file that changed but was not added]
  no changes added to commit (use "git add" and/or "git commit -a")
# fix: git add Module0-Foundation/Session1, then commit
```

```
$ git commit                                         # committed without -m: nano opens
  GNU nano ...   /home/kali/student-YOUR-USERNAME/.git/COMMIT_EDITMSG
  # Please enter the commit message for your changes. Lines starting
  # with '#' will be ignored, and an empty message aborts the commit.
  [the rest of the file, and nano's two lines of shortcuts]
# fix: type the message, save with Ctrl+O and Enter, leave with Ctrl+X
```

- **A commit that worked names its ID**, like `[main fbb04f9]`. Anything else means nothing was saved.
- **Read Git's message before you type again**: it names the problem, and usually the fix.

Common Mistakes: Pushing and Pulling

```
$ git status                              # committed, never pushed
  On branch main
  Your branch is ahead of 'origin/main' by 1 commit.
    (use "git push" to publish your local commits)
  nothing to commit, working tree clean
# fix: git push
```

```
$ git status                              # Git run outside your repository
  fatal: not a git repository (or any of the parent directories): .git
# fix: cd ~/student-YOUR-USERNAME first
```

```
$ git push                                # pushed after an edit on github.com
  To github.com:ORG/student-YOUR-USERNAME.git
   ! [rejected]        main -> main (fetch first)
  [6 lines: an error, and hints to pull first]
# fix: git pull, then git push: the pull puts GitHub's commit under yours
```

```
$ git pull                                # a worksheet filled in inside course-materials
  error: cannot pull with rebase: You have unstaged changes.
  error: Please commit or stash them.
# fix: copy it into your repository, then git restore . and git pull
```

- **Only a push reaches GitHub**, and only GitHub's copy is graded.
- **Never edit inside course-materials**: copy the worksheets out first.

Key Takeaways

- **One environment, inside Kali**, so every instruction works the same way for every student.
- **A worksheet is plain markdown**: what you type is what gets graded, so read it in Obsidian's reading view before you hand it in.
- **A screenshot is evidence**: the prompt, the command and the output in one capture, saved where the worksheet's link looks.
- **A commit stays on your computer until you push it**, and only GitHub's copy counts at the deadline.
- **Your key proves who you are to GitHub**, so an SSH URL needs no password, and a secret you commit stays in the history.
- **Read what Git prints**: `git status` says where your work is, and every error names its fix.

How the Practical Guide Uses This

1. **The Practical Guide is what you set up from**: the four tools inside Kali, Obsidian's link settings, Git's one-time settings, and your key on GitHub, with the exact steps rather than the reasons, and its worksheet records that each one works.
2. **In class, you hand in that worksheet and your Session 1 worksheets** through the whole cycle, with help at hand: pull, copy, add, commit, push.
3. **The Home Task is a small writing task you hand in alone, at home**: the same cycle a second time, with nobody watching.

Two documents sit beside you while you work rather than being read through: the **Tooling Reference** covers every tool used here in full, and the **Troubleshooting FAQ** is organized by the symptom in front of you, every mistake shown today included, with Git's full message.

Open the Practical Guide

`Module0_Session2_Practical_Guide.md`

Module 0 — Session 2
The Working Environment and the Submission Workflow